Table des matières

Table des matières 1
Structure générale d'un algorithme Structure générale d'un programme C# 3
Les types de données 4
Les déclarations 5
Les opérateurs arithmétiques 6
Ça peut servir… : Générer un nombre aléatoire 6
Afficher un résultat 7
Ça peut servir… : Formater un nombre à l'affichage 7
Saisir une valeur 8
Un point technique en C#… Quelle est la différence entre Convertir, Caster et Parser ? 9
Les structures alternatives 11
Les opérateurs de comparaison 12
Les opérateurs booléens 12
Les structures itératives 13
Ecrire une procédure 14
Ecrire une fonction 15
Procédures et fonctions (suite) : Passage par référence 16
Déclarer un tableau à 1 dimension 17

Parcourir un tableau à 1 dimension 18
Déclarer un tableau à 2 dimensions 19
Parcourir un tableau à 2 dimensions 20
Gérer les fichiers texte : Déclaration et mode d'ouverture 21
Gérer les fichiers texte : Lire et écrire les enregistrements 22
Créer une structure 23
Petit topo sur les chaînes de caractères 24
Les structures génériques de collections : Liste<T> 27
Les structures génériques de collections : Dictionnaire<TClé,TValeur> 29
Les structures génériques de collections : ListeChainee<T> 31
Les structures génériques de collections : Pile<T> 33
Les structures génériques de collections : File<T> 34

Structure générale d'un algorithme

Algorithmique

```
Algorithme Exemple
Déclarations
   //Déclaration à réaliser ici…

Début
   //Actions de l'algorithme ici…
Fin
```

Structure générale d'un programme C#

Langage C#

```
using System;
//bibliothèque qui apporte les éléments de base

class Exemple  //Enveloppe du programme
{
    static void Main()     //Procédure principale
    {
          //Déclarations à réaliser ici (bonne pratique)…

          //Actions du programme ici…
    }
}
```

Les types de données

Algorithmique	Langage C#
entier	short entier signé codé sur 2 octets int entier signé codé sur 4 octets long entier signé codé sur 8 octets
réel	float réel de 7 décimales codé sur 4 octets double réel de 15 décimales codé sur 8 octets decimal réel de 28 décimales codé sur 16 octets
caractère	char
chaîne de caractère (ou chaine)	string type spécial qui nécessitera plus d'explications…
booléen	bool

Remarque pour les réels en C# :

Pour préciser qu'un nombre est de type float, double ou décimal il faut rajouter les lettres f, d ou m :

```
0.57f              indique que 0.57 est un float
0.57d (ou 0.57)    indique que 0.57 est un double (par défaut)
0.57m              indique que 0.57 est un decimal
```

Les déclarations

Algorithmique	Langage C#

```
Algorithme EauxMinerales
Déclarations
    Constantes
          PRIX_UNITAIRE = 0.57 : réel

    Variables
          nomClient : chaîne de caractères
          nbBouteilles : entier
          montant, livraison, total : réel
Début
    …
```

```
//A Effectuer directement dans le Main

class EauxMinerales
{
    static void Main()
    {
        //Déclaration des constantes
        //La valeur est affectée directement
        //et de façon définitive
        const float PRIX_UNITAIRE = 0.57f;
         //f signifie que 0.57 est de type float

        //Déclaration des variables
        string nomClient;
        int nbBouteilles;
        float montant, livraison, total;
```

Remarque : Aussi bien en algorithmique qu'en C#, on utilise les conventions de nommage UPPER_CASE pour les constantes, camelCase pour les variables

Les opérateurs arithmétiques

Algorithmique	Exemple	Langage C#	Exemple
+ addition	totalNote = noteA + noteB	+	totalNote = nateA + noteB;
- soustraction	total = prix - remise	-	total = prix – remise;
* multiplication	noteCoef = note * coef	*	noteCoef = note * coef;
/ division	moyenne = total / nombre	/	moyenne = total / nombre;
DIV division euclidienne	moitie = nombre DIV 2	N'existe pas !	int moitie = (int)(nombre / 2); *//Il s'agit d'un cast en entier, seule la partie entière du calcul sera affectée*
MOD modulo (reste de la division euclidienne)	reste = nombre MOD 2	%	reste = nombre % 2;

Ça peut servir… : Générer un nombre aléatoire

Algorithmique

```
Déclarations
      Variables
            nombre1, nombre2 : entier
Début
      nombre1 = nombre aléatoire entre 0 et 9

      nombre2 = nombre aléatoire entre 2 et 9

Fin
```

Langage C#

Il faut utiliser une fonction fournie par le langage

```
int nombre1, nombre2;
Random rnd = new Random();  //Préparation
nombre1 = rnd.Next(10);     //Génère un nombre entre 0 et 9
                            //et l'affecte à nombre1

nombre2 = rnd.Next(2,10);   //Génère un nombre entre 2 et 9
                            //et l'affecte à nombre2
```

Afficher un résultat

Algorithmique

```
Algorithme Afficher
Déclarations
      Constantes
            MOIS = "septembre" : chaine

      Variables
            jour : entier
Début
      jour = 15

      Afficher "Le" + jour + MOIS + 2026
      Afficher "Le△" + jour + "△" + MOIS + "△2026"
      Afficher "Le△" + jour + "△" + MOIS + "△" + (2024 +
2)
```

Remarques : Le + est ici utilisé non pas comme opérateur arithmétique mais comme opérateur de concaténation de chaines.

Langage C#

```
const string MOIS = "septembre";
int jour = 15;

  //on peut faire comme dans l'algorithmique

Console.WriteLine("Le" + jour + MOIS + 2026);
Console.WriteLine("Le " + jour + " " + MOIS + " 2026");
Console.Writeline("Le " + jour + " " + MOIS + " " + (2024 + 2));


  //Ou utiliser une méthode plus spécifique à C#

Console.WriteLine($"Le {jour}{MOIS} 2026");

  //Entre {}, je place directement la variable (ou la constante)
  qui sera remplacée lors de l'affichage par la valeur contenue.
  Ne pas oublier de placer le caractère $ avant la chaine pour
  permettre cela.
```

Ça peut servir… : Formater un nombre à l'affichage

Algorithmique

```
prix = 20.5874f
Afficher prix + " €"        //2 décimales
```

Langage C#

```
float prix = 20.5874f;
Console.WriteLine(prix.ToString("0.00") + " €");
// ou bien  Console.WriteLine(prix.ToString("0.00 €"));
```

Saisir une valeur

Algorithmique	Langage C#

```
Algorithme Saisir
Déclarations
      Variables
            nomClient : entier
            nbBouteilles : entier
Début
      //Saisir du nom du client
      Afficher "Saisir le nom du client : "
      Saisir nomClient

      //Saisir le nombre de bouteilles
      Afficher "Quel est le nombre de bouteilles : "
      Saisir nbBouteilles
```

```
  //Déclarations
string nomClient;
int nbBouteilles;

  //Saisir du nom du client
Console.Write("Saisir le nom du client : ");
nomClient = Console.ReadLine();

  //Si l'on souhaite saisir autre chose que du texte (int, float,
  double, …) il faut effectuer une conversion de la chaine saisie
  grâce aux méthodes Parse ou Convert :

  //Saisir le nombre de bouteilles
Console.Write("Quel est le nombre de bouteilles : ");
nbBouteilles = int.Parse(Console.ReadLine());
  // ou
nbBouteilles = Convert.ToInt32(Console.ReadLine());


  //Si l'on veut s'assurer que ce qui est saisi est bien un int
  (par exemple) on peut utiliser la méthode TryParse :
bool valide;
valide = int.TryParse(Console.ReadLine(), out nbBouteilles);

  //Je tente de convertir en int, si ça fonctionne j'affecte la
  valeur à la variable nbBouteilles et true à valide, si ça ne
  fonctionne pas j'affecte false à valide.
```

Un point technique en C#… Quelle est la différence entre Convertir, Caster et Parser ?

Ces 3 notions sont souvent confondues, alors qu'elles sont bien différentes.

- **Convertir**

  - **Conversion implicite** : Consiste à affecter la valeur d'un contenant plus petit dans un contenant plus grand entre types compatibles. Cette opération est considérée comme sécurisée par le compilateur elle ne risque pas de provoquer d'erreur d'exécution.

```
int nombre = 521;
float nombreDecimal = nombre ;          //la valeur devient 521.0f
```

  - **Conversion explicite (Cast)** : Si la conversion peut aboutir à une perte de précision, il faut effectuer un Cast (ou transtypage). Cette opération consiste à prendre une valeur et tenter de lui changer de type. On effectue un Cast en plaçant le type cible entre parenthèses à côté de la valeur que l'on souhaite transtyper **(typeCible)valeur**. Le compilateur C# autorise de nombreux transtypages, par exemple, on peut transtyper un int en double, un char en int ou un float en decimal... Cette opération n'est pas totalement sécurisée car elle peut entrainer une perte de précision ou un résultat inattendu dû au fait qu'elle ne provoque pas d'exception.

```
double nombreDouble = 5.99999d;
int nombre = (int)nombreDouble;    //la valeur est maintenant 5 (partie décimale perdue)

char car = 'a';
int valueA = (int)car;             //la valeur est maintenant 97 (valeur unicode de 'a')

float nombreFlottant = 4.56f;
decimal nombreDecimal = (decimal)nombreFlottant;  //la valeur est maintenant 4.56m

int a = 300;
byte b = (byte)a;    // b est affecté à 44 (??) (valeur inattendue ici car la plage de byte
                     // est [0 ;255], comme 300 est trop grand on a affecté 300 MOD 256 soit 44
```

- **Conversion avec utilisation de la classe Convert** : C# fournit la classe Convert qui permet de tenter de convertir entre elles toutes les valeurs des types primitifs (ceux qui commencent par une minuscule, int, char, string, …) et non forcément compatibles. Contrairement au Cast, Convert tente d'interpréter ou d'adapter la valeur afin de produire un résultat cohérent.

```
decimal uneSomme = 5.67m;
int uneAutreSomme = Convert.ToInt32(uneSomme);  //la valeur est maintenant 6 (ici un arrondi)

double uneValeur = 5.33;
string chaine = Convert.ToString(uneValeur);   //la valeur est maintenant "5.33" (types non compatibles)

int valeur = 0;
bool isFalse = Convert.ToBoolean(valeur);        //la valeur affectée est false (valeur interprétée),
                                                 // toute autre valeur que 0 serait interprétée à true

string chaine = "5.632";
~~decimal valeur = Convert.ToDecimal(chaine);~~      // provoque une exception de type FormatException
```

- **Parser** : Consiste à analyser une chaîne de caractères pour la convertir vers un type primitif. C# fournit les méthodes Parse et TryParse pour la plupart des types primitifs.

```
string chaine = "5.632";
decimal valeur = decimal.Parse(chaine);                 // valeur est affectée à 5.632m

string value = "5.0";
decimal result;
bool isValid = decimal.TryParse(value, out result);     // Si la conversion peut se faire
                                                        // isValid est affecté à true (false sinon)
                                                        // et result est affecté à 5.0m
```

Les structures alternatives

Algorithmique	Langage C#

```
Si condition Alors
    //Actions si condition évaluée à vrai
FinSi
```

```
Si condition Alors
    //Actions si condition évaluée à vrai
 Sinon
    //Actions si condition évaluée à faux
FinSi
```

```
Selon choix
    Cas 1:
        //Action si choix == 1
    Cas 2 :
        //Action si choix == 2
    Cas Sinon
        //Action si choix <> 1 et 2
FinSelon
```

```
if(condition)
{
    //Actions si condition évaluée à true
}
```

```
if(condition)
{
    //Action si condition évaluée à true
}
else
{
    //Action si condition évaluée à false
}
```

```
switch(choix)
{
    case 1:
        //Action si choix == 1
        break;
    case 2:
        //Action si choix == 2
        break;
    default:
        //Action si choix != 1 et != 2
}
```

le break permet de sortir du switch sans continuer au case 2

Les opérateurs de comparaison

Algorithmique		Langage C#		
== égal	Si a == b Alors	==		if(a == b)
< strictement inférieur	Si a < b Alors	<		if(a < b)
> strictement supérieur	Si a > b Alors	>		if(a > b)
<= inférieur ou égal	Si a <= b Alors	<=		if(a <= b)
>= supérieur ou égal	Si a >= b Alors	>=		if(a >= b)
<> différent	Si a <> b Alors	!=	//qui signifie NOT equal	if(a != b)

Les opérateurs booléens

Algorithmique		Langage C#		
				Idem if(fini == false)
NON	Si NON fini Alors	!	(NOT)	if(!fini)
ET	Si age < 10 ET dep == 67 Alors	&&	(AND)	if(age < 10 && dep == 67)
OU	Si dep == 67 OU dep == 68 Alors	||	(OR)	if(dep == 67 || dep == 68)
				touche du clavier **Alt Gr 6**

Les structures itératives

Algorithmique	Langage C#

```
TantQue condition
      //Actions à répéter
FinTantQue
```

```
while(condition)
{
      //Actions à répéter
}
```

```
Pour i de 0 à 10            (Pas 1 par défaut)
      //Actions à répéter
FinPour
```

Valeur départ — valeur arrivée — Pas

```
for(i = 0; i <= 10 ; i++)
{
      //Actions à répéter
}
```

```
Faire
      //Actions à répéter
TantQue condition
```

```
do
{
      //Actions à répéter
} while(condition);
```

```
//La structure Pour Chaque servira à parcourir
//une collection (à partir des structures génériques
//de collection List<T>)
```

```
//La structure foreach() permettra de parcourir
//des collections
```

Ecrire une procédure

Algorithmique

```
Procédure afficherBanniere(version : chaîne)
Déclarations
Début
      Afficher "-----------------------------------"
      Afficher "|  Top Programme" + version + "     |"
      Afficher "-----------------------------------"
Fin



Algorithme topProgramme
Déclarations
      Variables
            versionDate : chaîne
Début
      versionDate = "V5.4 – Sept 2026"

      //Appel de la procédure
      afficherBanniere(versionDate)
      …
      afficherBanniere(versionDate)
Fin
```

Nom de procédure en camelCase

Langage C#

```
class topProgramme          //Programme principal
{
    static void Main()
    {
            //Déclarations
            string versionDate;

            versionDate = "V5.4 – Sept 2026";

            //Appel de la procédure
            AfficherBanniere(versionDate);
            …
            AfficherBanniere(versionDate);
     }

     //Codage de la Procédure
    static void AfficherBanniere(string version)
    {
      Console.WriteLine("---------------------------------");
      Console.WriteLine("| Top Programme" + version + " |";
      Console.WriteLine("---------------------------------"
    }
}
```

C'est le mot clé **void** "retour vide" qui indique qu'il s'agit d'une **procédure**

Nom de procédure en PascalCase

Ecrire une fonction

Algorithmique

```
Fonction francsToEuros(montantFr : réel)
Type de sortie : Réel
Déclarations
      Constantes
            VAL_EURO = 6.55956 : réel
      Variables
            montantEu : réel
Début
      montantEu = montantFr / VAL_EURO
      Retourner montantEu
Fin
```

```
Algorithme conversionF_E
Déclarations
      Variables
            montant, resultat : réels
Début
      Afficher "Saisir le montant en francs : "
      Saisir montant
      //Appel de la fonction
      resultat = francsToEuros(montant)
      Afficher montant + " F = " + resultat + " €"
Fin
```

Algo : nom de fonction en camelCase

Langage C#

```
class conversion_E          //Programme principal
{
    static void Main()
    {
            //Déclarations
            float montant, resultat;

            Console.Write("Saisir le montant en francs : ")
            montant = float.Parse(Console.ReadLine());

            //Appel de la fonction
            resultat = FrancsToEuros(montant);

            Console.WriteLine($"{montant} F = {resultat} €");
     }

     //Codage de la fonction
    static float FrancsToEuros(float montantFr)
    {
            const float VAL_EURO = 6.55956f;
            float montantEu;
            montantEu = montantFr * VAL_EURO;
            return montantEu; //retour de la valeur
    }
}
```

Le mot clé **float** (type autre que void) indique qu'il s'agit d'une **fonction** et que la valeur retournée par cette fonction sera obligatoirement de type float

C# : nom de fonction en PascalCase

Procédures et fonctions (suite) : Passage par référence

Algorithmique

```
Algorithme exempleRef
Déclarations
      Variables
            unNombre : entier
Début
      unNombre = 67

      Afficher "Avant : " + unNombre

      //Appel de la procédure
      plusDix(référence unNombre)

      Afficher "Après : " + unNombre
Fin



Procédure plusDix(référence nombre : entier)
Déclarations
Début
      nombre = nombre + 10
Fin
```

Langage C#

```
class exempleRef              //Programme principal
{
    static void Main()
    {
            //Déclarations
            int unNombre;

            unNombre = 67;
            Console.WriteLine("Avant : " + unNombre);

            //Appel de la procédure
            PlusDix(ref unNombre);

            Console.WriteLine("Après : " + unNombre);
     }

      //Codage de la procédure
    static void PlusDix(ref int nombre)
    {
            nombre = nombre + 10;
    }
}
```

Déclarer un tableau à 1 dimension

Algorithmique

```
Algorithme testTableau
Déclarations
      Variables
            lesNotes : tableau de 5 réels
Début
…
Fin
```

```
//Avec initialisation dès la déclaration
Algorithme testTableau
Déclarations
      Variables
            lesNotes : tableau de 5 réels initialisé
                        à {11,7,14.5,10,14}
Début
…
Fin
```

Langage C#

```
class testTableau
{
    static void Main()
    {
            //1-Déclarations
            float[] lesNotes; //tableau de float

            //2-Création du tableau
            lesNotes = new float[5];
      }
}
```

je crée un tableau composé de 5 cases. Chaque case stockera un float

```
class testTableau
{
    static void Main()
    {
            //Déclaration et initialisation
            float[] lesNotes = {11,7,14.5f,10,14};
      }
}
```

Dans ce cas, inutile de faire le new, le tableau est créé avec le nombre de cases égal au nombre d'éléments de la liste (ici 5).

Parcourir un tableau à 1 dimension

Algorithmique

```
Pour i de 0 à 4
      Afficher lesNotes[i]
FinPour



// Connaître la taille du tableau : taille(tab)

//Parcours du tableau en entier
Pour i de 0 à taille(tab) - 1
      Afficher lesNotes[i]
FinPour
```

Langage C#

```
for(int i = 0; i <= 4 ; i++)
{
      Console.WriteLine(lesNotes[i]);
}


//Connaître la taille du tableau : lesNotes.Length

//Parcours du tableau en entier
for(int i = 0; i <= lesNotes.Length - 1 ; i++)
{
      Console.WriteLine(lesNotes[i]);
}

//Ou bien
for(int i = 0; i < lesNotes.Length ; i++)
{
      Console.WriteLine(lesNotes[i]);
}
```

Déclarer un tableau à 2 dimensions

Algorithmique

```
Algorithme testTableau
Déclarations
      Variables
            lesNotesAnnee : tableau de 3 lignes et
                            5 colonnes de réels
Début
…
Fin
```

```
//Avec initialisation dès la déclaration
Algorithme testTableau
Déclarations
      Variables
            lesNotesAnnee : tableau de 3 lignes et 5
colonnes de réels initialisé à {{11,7,14.5,10,14},
                                {10,7.5,15,09,14},
                                {1,7,12.5,11,13}}
Début
…
Fin
```

Langage C#

```
class testTableau
{
    static void Main()
    {
        //1-Déclarations
        float[,] lesNotesAnnee;     //tableau 2D de float

        //2-Création du tableau
        lesNotesAnnee = new float[3,5];
    }
}
```

je crée un tableau composé
de 3 lignes et 5 colonnes
Chaque case stockera un float

```
class testTableau
{
    static void Main()
    {
        //Déclaration et initialisation
        float[,] lesNotesAnnee = {{11,7,14.5f,10,14},
                                  {10,7.5f,15,09,14},
                                  {1,7,12.5f,11,13}};
    }
}
```

Dans ce cas, inutile de faire le new, le tableau
est créé avec le nombre de lignes et colonnes
correspondant aux valeurs d'initialisation.

Parcourir un tableau à 2 dimensions

Algorithmique

```
Pour i de 0 à 2              //Parcours des lignes
      Pour j de 0 à 4        //Parcours des colonnes
            Afficher lesNotesAnnee[i,j]
      FinPour
FinPour



//Connaître le nombre de lignes du tableau :
            tailleLignes(lesNotesAnnee)


//Connaître le nombre de colonnes du tableau :
            tailleColonnes(lesNotesAnnee)


Pour i de 0 à tailleLignes(lesNotesAnnee) - 1
      Pour j de 0 à tailleColonnes(lesNotesAnnee) - 1
            Afficher lesNotesAnnee[i,j]
      FinPour
FinPour
```

Langage C#

```
for(i = 0; i <= 2 ; i++)          //Parcours des lignes
{
      for(j = 0; j <= 4; j++)     //Parcours des colonnes
      {
            Console.WriteLine(lesNotesAnnee[i,j]);
      }
}



      lesNotesAnnee.GetLength(0)  //1ère dimension (lignes)



      lesNotesAnnee.GetLength(1)  //2ème dimension (colonnes)


for(i = 0; i <= lesNotesAnnee.GetLength(0) - 1 ; i++)
{
      for(j = 0; j <= lesNotesAnnee.GetLength(1) - 1; j++)
      {
            Console.WriteLine(lesNotesAnnee[i,j]);
      }
}
```

Gérer les fichiers texte : Déclaration et mode d'ouverture

Algorithmique

```
Déclarations
      Variables
            lesEtudiants : fichier séquentiel
            ligne : chaîne
Début

      Si existe("lesEtudiants.txt") Alors
            Ouvrir lesEtudiants associé à
"lesEtudiants.txt" en mode Lecture


            ou en mode :      Ajout
                              Ecriture
```

Langage C#

```
  //Nécessite de rajouter la bibliothèque System.IO;
using System.IO;


if(File.Exists("lesEtudiants.txt"))
{
      //Ouverture en Lecture
      //En 2 temps
      //1-on crée un flux qui permet de lire des fichiers…
      FileStream fs = new FileStream("lesEtudiants.txt",
            FileMode.Open, FileAccess.Read);
      //2-Ce flux sera de type texte
      StreamReader lesEtudiants = new StreamReader(fs);
}


  //Ouverture en mode Ajout
      FileStream fs = new FileStream("lesEtudiants.txt",
            FileMode.Append, FileAccess.Write);
      StreamWriter lesEtudiants = new StreamWriter(fs);


  //Ouverture en mode écriture (fichier vidé préalablement)
      FileStream fs = new FileStream("lesEtudiants.txt",
            FileMode.Create, FileAccess.Write);
      StreamWriter lesEtudiants = new StreamWriter(fs);
```

Gérer les fichiers texte : Lire et écrire les enregistrements

Algorithmique	Langage C#
`//lire un enregistrement` `ligne = Lire lesEtudiants`	`string ligne;` `ligne = lesEtudiants.ReadLine();`

`//lire tous les enregistrements en testant`
`// la fin de fichier`
`TantQue NON finDeFichier(lesEtudiants)`
`  ligne = Lire lesEtudiants`
`FinTantQue`
	`while(!lesEtudiants.EndOfStream)`
	`{`
	`  ligne = lesEtudiants.ReadLine();`
	`}`

`//Ecrire un enregistrement`
`//après avoir ouvert le fichier en mode`
`//Ajout ou en mode Ecriture`
`ligne = "20;Davis;Miles";`
`Ecrire lesEtudiants, ligne`
	`ligne = "20;Davis;Miles";`
	`lesEtudiants.WriteLine(ligne);`

`// Fermer le fichier` `Fermer lesEtudiants`	`lesEtudiants.Close();`

Créer une structure

Algorithmique

```
Algo testStrut

      Structure Etudiant     //Par convention en PascalCase
            nom : chaîne
            prenom : chaine
            groupe : entier
            resultats : tableau de 6 réels

            //Constructeur
            procédure Etudiant (nm : chaine, pn : chaine,
                                grp : entier)
            Début
                  nom = nm
                  prenom = pn
                  groupe= grp
            Fin
      FinStructure

Déclarations
      Variable
            et : Etudiant

Début

      //Création d'un nouvel étudiant
      et = new Etudiant("Zappa", "Frank", 3)

      Afficher et.prenom + " " + et.nom;
```

Langage C#

```
class TestStruct
                                          //Obligatoire
      struct Etudiant
      {
            public string nom;
            public string prenom;
            public int groupe;
            public float[] resultats;

            //Constructeur
            public Etudiant(string nm, string pn, int grp)
            {
                  nom = nm;
                  prenom = pn;
                  groupe = grp;
                  //Construction du tableau ici !
                  resultats = new float[6];
            }
      }

      public static void Main()
      {
            //Déclaration
            Etudiant et;
            //Création d'un nouvel étudiant
            et = new Etudiant("Zappa", "Frank", 3);
            Console.WriteLine(et.prenom + " " + et.nom);
```

Petit topo sur les chaînes de caractères

Algorithmique	Langage C#

```
Déclarations
      Variables
            message : chaine
Début
      message = "Bonjour"

      //Connaître la longueur
      Afficher longueur(message)  //Affichera 7

      //Accéder à un caractère par son indice
      //(1er indice à 0)
      Afficher message[5]         //Affichera 'u'

      //Parcourir une chaîne dans une itération
      Pour i de 0 à longueur(tabMessage) - 1
            Afficher message[i]
      FinPour

      //Vérifier si la chaine contient une sous-chaine
      //à ne pas confondre avec du chouchen ;-)
      Afficher contient(message, "ou")       //Affichera Vrai
      Afficher contient(message, "fz")       //Affichera Faux

      //Connaître l'indice de début d'une sous-chaîne
      indiceDe(message, "ou")     //Affichera 4
      indiceDe(message, "fz")     //Affichera -1
```

```
String message;

message = "Bonjour";


Console.WriteLine(message.Length);



Console.WriteLine(message[5]);


for(int i = 0; i < message.Length; i++)
{
      Console.Write(message[i]);
}


Console. WriteLine(message.Contains("ou"));
Console. WriteLine(message.Contains("fz"));


Console. WriteLine(message.IndexOf("ou"));
Console. WriteLine(message.IndexOf("fz"));
```

Petit topo sur les chaînes de caractères (suite)

Algorithmique	Langage C#

```
//Récupérer une sous-chaîne (toujours pas du chouchen ;-))
      // Ne modifie pas la chaine d'origine !
      //Paramètre 1 : chaîne dans laquelle rechercher
      //Paramètre 2 : indice de début – commence à 0
      //Paramètre 3 : nombre de caractères à récupérer
      Afficher sousChaine(message, 0, 3)   //Affichera "Bon"
```

```
Console. WriteLine(message.Substring(0,3));
```

```
      //Récupérer une version en majuscule de la chaine
      // Ne modifie pas la chaine d'origine !
      Afficher enMajuscule(message)     //Affichera "BONJOUR"
```

```
Console. WriteLine(message.ToUpper());
```

```
      //Récupérer une version en minuscule de la chaine
      // Ne modifie pas la chaine d'origine !
      Afficher enMinuscule(message)     //Affichera "bonjour"
```

```
Console.WriteLine(message.ToLower());
```

```
      //Vérifier si la chaine commence par cette sous-chaine
      Afficher commencePar(chaine, "Bon")     //Affichera Vrai
```

```
Console.WriteLine(message.StartsWith("Bon"));
```

```
      //Comparer 2 chaines
      Si message == "Bonjour" Alors
            Afficher "Egal"
      Finsi
```

```
if(message == "Bonjour")
{
      Console.WriteLine("Egal");
}
```

```
      //Concaténer 2 chaines (opérateur de concaténation +)
      message = message + " à tous !"
      Afficher message       //Affichera "Bonjour à tous !"
```

```
//Fonctionne mais très gourmant car création
//d'un nouvel espace mémoire (voir la classe
//StringBuilder)
message = message + " à tous";
Console.WriteLine(message);
```

Petit topo sur les chaînes de caractères (suite et fin)

```
   //Diviser une chaîne au format CSV (ou autre) autour d'un
séparateur (ou délimiteur)


Déclarations
      Variables
            ligne : chaîne
            temp : tableau de chaînes

Début

      ligne = "1;Yakamoto;Claire;Y"
      temp = diviser(ligne,';')
```

Cette action va remplir automatiquement le tableau temp :

	0	1	2	3
temp	"1"	"Yakamoto"	"Claire"	"Y"

```
string ligne;
string[] temp;

//Déclaration du séparateur
char[] separateur = new char[] {';'};

ligne = "1;Yakamoto;Claire;Y";
temp = ligne.Split(separateur);
```

Les structures génériques de collections : Liste<T>

Algorithmique	Langage C#
	`using System.Collections.Generic; //Obligatoire`
`Algo testListe`	
`Déclarations`	`class TestList`
`  Variable`	`{`
`  lesMusiciens : Liste<chaine>`	`  //Déclaration`
	`  List<string> lesMusiciens;`
`Début`	`  //Création`
	`  lesMusiciens = new List<string>();`
`  //Ajouter en fin de liste`	
`  lesMusiciens.Ajouter("John Coltrane")`	`  lesMusiciens.Add("John Coltrane");`
`  lesMusiciens.Ajouter("Elvin Jones")`	`  lesMusiciens.Add("Elvin Jones");`
`  //Insérer avec un indice`	
`  lesMusiciens.Insérer(1,"McCoy Tyner")`	`  lesMusiciens.Insert(1,"McCoy Tyner");`
`  lesmusiciens.Insérer(2,"Jimmy Garrison")`	`  lesMusiciens.Insert(2,"Jimmy Garrison");`
`  //Retirer un item (1ère occurrence)`	
`  lesMusiciens.Retirer("Elvin Jones")`	`  lesMusiciens.Remove("Elvin Jones");`
`  //Retirer l'item situé à un indice`	
`  lesMusiciens.RetirerA(1)`	`  lesMusiciens.RemoveAt(1);`
`  //Vérifier si la liste contient un item`	
`  Afficher lesMusiciens.Contient("John Coltrane")`	`  Console.WriteLine(lesMusiciens.Contains("John Coltrane");`
`  //Obtenir l'indice d'un Item`	
`  Afficher lesMusiciens.IndiceDe("John Coltrane")`	`  Console.WriteLine(lesMusiciens.IndexOf("John Coltrane");`

```
//Accéder à un item par son indice
Afficher lesMusiciens[0]

//Connaitre la taille de la liste
Afficher lesMusiciens.nbItem

//Trier la liste par ordre croissant
lesMusiciens.Trier()

//Renverser la liste
lesMusiciens.Renverser()


//Parcourir la liste entièrement
//la variable musicien doit être déclarée
// du même type T que la liste !

Pour chaque musicien Dans lesMusiciens
     Afficher musicien
FinPour
```

```
Console.WriteLine(lesMusiciens[0]);

Console.WriteLine(lesMusiciens.Count);

lesMusiciens.Sort();

lesMusiciens.Reverse();

foreach(string musicien in lesMusiciens)
{
     Console.WriteLine(musicien + "-");
}
```

Les structures génériques de collections : Dictionnaire<TClé,TValeur>

Algorithmique	Langage C#

```
Algo testDico
Déclarations
      Variable
            lesInstrus : Dictionnaire<chaine,chaine>
            musicien, instru : chaine
Début

      //Ajouter en fin de liste
      lesInstrus.Ajouter("John Coltrane", "saxophone")
      lesInstrus.Ajouter("Elvin Jones","drums")
      lesInstrus.Ajouter("McCoy Tyner","piano")
      lesInstrus.Ajouter("Jimmy Garrison", "bass")


      //Retirer un item avec sa clé
      lesInstrus.Retirer("Elvin Jones")


      //Vérifier si une clé est déjà présente
      Afficher lesInstrus.ContientClé("John Coltrane")


      //Vérifier si une valeur est présente
      Afficher lesInstrus.ContientValeur("bass")

      //Accéder à un item par sa clé
      Afficher lesInstrus["Elvin Jones"]
```

```
using System.Collections.Generic; //Obligatoire

class TestDico
{
      //Déclaration du dictionnaire <TKey, TValue>
      Dictionary<string, string> lesInstrus;
      //Création
      lesInstrus = new Dictionary<string,string>();

      lesInstrus.Add("John Coltrane","saxophone");
      lesInstrus.Add("Elvin Jones","drums");
      lesInstrus.Add("McCoy Tyner","piano");
      lesInstrus.Add("Jimmy Garrison","bass");


      lesInstrus.Remove("Elvin Jones");


      Console.WriteLine(lesInstrus.ContainsKey("John Coltrane");


      Console.WriteLine(lesInstrus.ContainsValue("bass");


      Console.WriteLine(lesInstrus["Elvin Jones"]);
```

```
//Connaitre la taille du dictionnaire
Afficher lesInstrus.nbItem
```

```
   Console.WriteLine(lesInstrus.Count);
```

```
//Parcourir le dictionnaire entièrement
//Il faut parcourir la collection des clés
//la variable musicien doit être déclarée
//du même type TClé que la clé

Pour chaque musicien Dans lesInstrus.Clés
 Afficher musicien + " : " + lesInstrus[musicien]
FinPour
```

```
foreach(string musicien in lesInstrus.Keys)
{
     Console.WriteLine(musicien + " : " + lesInstrus[musicien]);
}
```

```
//Parcourir les valeurs du dictionnaire
//Il faut parcourir la collection des valeurs
//la variable instru doit être déclarée
//du même type TValeur que la valeur

Pour chaque instru Dans lesInstrus.Valeurs
      Afficher instru
FinPour
```

```
foreach(string instru in lesInstrus.Values)
{
  Console.WriteLine(instru);
}
```

Les structures génériques de collections : ListeChainee<T>

Algorithmique	Langage C#

```
Algo testListeChainee
Déclarations
      Variable
            laSalleAttente : ListeChainee<chaine>
            nd, enCours : NoeudListeChainee<chaine>

Début

      //Ajouter en début ou fin de liste
      laSalleAttente.AjouterPremier("John Coltrane")
      laSalleAttente.AjouterDernier("Elvin Jones")

      //Ajouter après un nœud
      nd = laSalleAttente.Premier
      laSalleAttente.AjouterAprès(nd,"Jimmy Garrison")

      //Retirer le 1er noeud
      laSalleAttente.RetirerPremier()

      //Retirer le dernier noeud
      laSalleAttente.RetirerDernier()

      //Retirer la 1ère occurrence d'un élément
      laSalleAttente.Retirer("Elvin Jones")

      //Retirer un nœud existant
      nd = laSalleAttente.Premier
      laSalleAttente.Retirer(nd)
```

```
using System.Collections.Generic; //Obligatoire

class TestLinkedList
{
      //Déclaration
      LinkedList<string> laSalleAttente;
      LinkedListNode<string> nd, enCours;
      //Création
      laSalleAttente = new LinkedList <string>();

      laSalleAttente.AddFirst("John Coltrane");
      laSalleAttente.AddLast("Elvin Jones");


      nd = laSalleAttente.First;
      laSalleAttente.AddAfter(nd,"Jimmy Garrison");


      laSalleAttente.RemoveFirst();


      laSalleAttente.RemoveLast();


      laSalleAttente.Remove("Elvin Jones");


      nd = laSalleAttente.First;
      laSalleAttente.Remove(nd);
```

```
//Accès au 1er élément
Afficher laSalleAttente.Premier.Valeur
```

```
Console.WriteLine(laSalleAttente.First.Value);
```

```
//Accès au dernier élément
Afficher laSalleAttente.Dernier.Valeur
```

```
Console.WriteLine(laSalleAttente.Last.Value);
```

```
//Trouver le nœud de la 1ère occurrence d'un élément
nd = laSalleAttente.Trouver("John Coltrane")
```

```
nd = laSalleAttente.Find("John Coltrane");
```

```
//Connaitre la taille de la liste chainée
Afficher laSalleAttente.nbItem
```

```
Console.WriteLine(laSalleAttente.Count);
```

```
//Savoir si la liste chainée contient un élément
Afficher laSalleAttente.Contient("John Coltrane")
```

```
Console.WriteLine(laSalleAttente.Contains("John Coltrane");
```

```
//Vider la liste chainée
laSalleAttente.Vider()
```

```
laSalleAttente.Clear();
```

```
//Parcourir la liste chainée entièrement
enCours = laSalleAttente.Premier
TantQue enCours <> null
      Afficher enCours.Valeur
      //Aller au nœud suivant
      enCours = enCours.Suivant
FinTantQue
```

```
LinkedListNode<string> enCours;
enCours = laSalleAttente.First;
while(enCours != null)
{
      Console.WriteLine(enCours.Value);
      enCours = enCours.Next;
}
```

Les structures génériques de collections : Pile<T>

Algorithmique	Langage C#
`Algo testPile`	`using System.Collections.Generic;  //Obligatoire`
`Déclarations`	
`  Variable`	`class TestStack`
`    unePile : Pile<entier>`	`{`
`    nb : entier`	`  Stack<int> unePile;  //Déclaration`
	`  int nb;`
`Début`	`  unePile = new Stack<int>();  //Création`
`  //Empiler`	
`  unePile.Empiler(22)`	`  unePile.Push(22);`
`  //Atteindre le sommet sans le dépiler`	
`  nb = unePile.Sommet()`	`  nb = unePile.Peek();`
`  //Dépiler`	
`  unePile.Depiler()`	`  unePile.Pop();`
`  //Connaitre la taille de la pile`	
`  Afficher unePile.nbItem`	`  Console.WriteLine(unePile.Count);`
`  //Savoir si la pile contient un élément`	
`  Afficher unePile.Contient(22)`	`  Console.WriteLine(unePile.Contains(22));`
`  //Vider la pile`	
`  unePile.Vider()`	`  unePile.Clear();`
`  //Parcourir et dépiler une pile`	`  while(unePile.Count > 0)`
`  TantQue unePile.NbItem > 0`	`  {`
`    Afficher unePile.Sommet()`	`    Console.WriteLine(unePile.Peek());`
`    unePile.Depiler()`	`    unePile.Pop();`
`  FinTantQue`	`  }`

Les structures génériques de collections : File<T>

Algorithmique	Langage C#
Algo testFile	using System.Collections.Generic; //Obligatoire
Déclarations	
Variables	class TestQueue
uneFile : File<entier>	{
nb : entier	**Queue<int>** uneFile; //Déclaration
Début	int nb;
//Enfiler	uneFile = **new Queue<int>();** //Création
unefile.**Enfiler**(22)	uneFile.**Enqueue**(22);
//Atteindre le sommet sans le défiler	
nb = unefile.**Sommet**()	nb = uneFile.**Peek**();
//Défiler	
unefile.**Defiler**()	uneFile.**Dequeue**();
//Connaitre la taille de la file	
Afficher unefile.**nbItem**	Console.WriteLine(uneFile.**Count**);
//Savoir si la file contient un élément	
Afficher unefile.**Contient**(22)	Console.WriteLine(uneFile.**Contains(22));**
//Vider la file	
unefile.**Vider**()	uneFile.**Clear**();
//Parcourir et défiler dans une file	while(uneFile.Count > 0)
TantQue unefile.NbItem > 0	{
Afficher unefile.Sommet()	Console.WriteLine(uneFile.**Peek**());
unefile.Defiler()	uneFile.**Dequeue**();
FinTantQue	}